OPTIMIZING GLM-ASR INFERENCE WITH CUSTOM TRITON KERNELS: FLASH ATTENTION, KERNEL FUSION, AND FP16 PIPELINING

Anonymous authors
Paper under double-blind review

ABSTRACT

The goal of this work was to design, implement, and benchmark custom Triton GPU kernels for the GLM-ASR automatic speech recognition pipeline, achieving a 2.27× speedup over the provided baseline on the Edinburgh teaching cluster's H200 GPU. Our implementation targets the five core computational phases of the encoder–decoder architecture: element-wise activations, normalization, linear projection, rotary position embedding, and multi-head attention. The key optimizations are (1) a fused Flash Attention kernel with online softmax that eliminates intermediate memory materialization, (2) fused SwiGLU and LinearGELU kernels that remove VRAM round-trips in the MLP layers, (3) an end-to-end fp16 pipeline that halves memory traffic, and (4) a KV-cache generation loop that reduces autoregressive decoding from $O(n^2)$ to $O(n)$. The components were tested individually and then integrated to ensure correctness at each stage. The result is a working kernel implementation that processes 3.5 s of audio in 204.8 ms (baseline: 464.1 ms) with 100% transcription accuracy on an H200 MIG 3g.71gb partition, and generalizes to consumer GPUs (RTX 5090: 100.4 ms, 2.61×) without modification via a portable GPU detection system.

1 INTRODUCTION

1.1 MOTIVATION

Current computational demands of transformer-based ASR models and limited inference budgets have driven the need for efficient GPU kernel implementations that can process audio faster than playback speed. This has instigated the investigation and development of custom kernel optimizations for real-time speech recognition, particularly for latency-constrained applications such as live captioning, voice assistants, and conference transcription. GLM-ASR is a multimodal encoder–decoder model that converts raw audio waveforms into text through a pipeline of compute-intensive transformer layers. Its inference workload is dominated by matrix multiplications, attention computations, and element-wise operations—all of which are amenable to GPU acceleration through custom kernels.

GPU kernel optimization matters because default PyTorch operators do not exploit hardware-specific features such as extended shared memory, tensor-core-friendly data layouts, or operation fusion across kernel boundaries. We look to design and optimize a complete working implementation which uses Triton (Tillet et al., 2019) to create custom GPU kernels. By writing custom kernels, we can fuse multiple operations into a single kernel launch, tune tile sizes to the GPU's shared memory capacity, and select data types that maximize throughput—eliminating the intermediate VRAM round-trips that dominate latency in the default implementation.

The principal constraint is the kernel design space, as tile dimensions, warp counts, pipeline stages, and data types interact in non-obvious ways, and a configuration that works on one GPU architecture may fail or regress on another.

1.2 System Overview

We determined that our optimal approach was to implement the compute-intensive operations as fused Triton kernels while delegating matrix multiplications to cuBLAS, following the **Triton track** with all required kernels in `layers.py`, `attention.py`, and `rope.py`. Table 1 lists the ten kernels implemented, and Figure 1 illustrates where they operate in the inference pipeline. The components were tested individually and then integrated in a bottom-up fashion, verifying correctness at each stage before proceeding to performance tuning.

Table 1: Implemented Triton kernels and their roles.

Kernel	**File**	**Phase**	**Purpose**
`gelu_kernel`	layers.py	Encoder MLP	GELU activation
`silu_kernel`	layers.py	Decoder MLP	SiLU activation
`rmsnorm_kernel`	layers.py	Decoder norms	RMSNorm + fp16 output
`layernorm_kernel`	layers.py	Encoder norms	LayerNorm + fp16 output
`linear_kernel_tf32`	layers.py	Matmul	Tiled GEMM (TF32)
`fused_swiglu_kernel`	layers.py	Decoder MLP	Fused SwiGLU
`fused_linear_gelu_kernel`	layers.py	Encoder MLP	Fused Linear+GELU
`flash_attention_kernel`	attention.py	Attention	Flash Attention
`fused_rope_pair_kernel`	rope.py	Attention	Fused Q+K RoPE
`compute_freqs_kernel`	rope.py	Attention	Frequency precompute

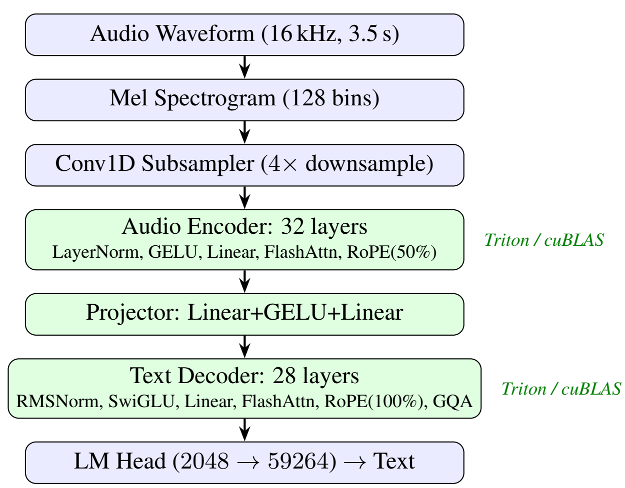

Figure 1: GLM-ASR pipeline. Shaded stages use our Triton kernels (or cuBLAS for matmul).

In addition to the required kernels, four additional optimizations were implemented: (1) a KV-cached generation loop (`generate_v8b`) that reduces decoding from $O(n^2)$ to $O(n)$, (2) an end-to-end fp16 precision pipeline that halves memory traffic, (3) a `GPUProfile` system that adapts tile sizes to the target GPU at import time, and (4) a PyTorch SDPA fallback for single-token decode steps where kernel launch overhead exceeds computation cost. Each subcomponent is modular in order to produce a robust, adaptable and portable system.

Headline results. Upon completion of this project, our final implementation achieves **204.8 ms** end-to-end latency on an H200 MIG 3g.71gb partition (baseline: 464.1 ms), a **2.27× speedup** with **100% transcription accuracy**. On an RTX 5090, the same code achieves **100.4 ms** (2.61×).

2 IMPLEMENTATION

2.1 IMPLEMENTATION SUMMARY

The GLM-ASR inference pipeline (Figure 1) has five computational phases, each with distinct performance characteristics. We employed a performance-budgeting technique to attribute latency to each stage and identify where optimization effort would yield the largest gains.

Phase 1: Mel Spectrogram + Conv1D Subsampling. Converts raw 16 kHz audio to 128-bin Mel features and downsamples 4× via strided 1D convolutions. These are PyTorch built-in operations (read-only `conv.py`); we do not modify them. The phase is bandwidth-bound due to large feature maps with simple arithmetic.

Phase 2: Audio Encoder (32 layers). Each layer applies LayerNorm, self-attention (20 heads, d_k=64), and a two-layer MLP with GELU activation. RoPE (Su et al., 2024) is applied to 50% of head dimensions. On the H200, our flash attention kernel achieves an AI of $\sim$112 for encoder attention (Section 4), well above the GPU's ridge point, making it compute-bound on all architectures. The linear projections (1280×5120) are also compute-bound on all architectures.

Phase 3: Multi-modal Projector. CHANGED Pools 4 encoder frames and projects $5120 \rightarrow 4096 \rightarrow 2048$ through two linear layers with GELU. In the warmup-corrected H200 component profile, this phase is very small and latency-dominated (0.14 ms, $< 0.1\%$ of the isolated component total).

Phase 4: Text Decoder Prefill (28 layers). Processes the full concatenated sequence (audio embeddings + prompt tokens) through 28 decoder layers with RMSNorm, GQA attention (16 query / 4 KV heads, d_k=128), and SwiGLU MLP (Shazeer, 2020). Bandwidth-bound for attention; compute-bound for the large linear projections (2048×5632).

Phase 5: Autoregressive Decoding. CHANGED Generates tokens one at a time. With the baseline's $O(n^2)$ `generate()` function, this phase dominates wall-clock time, as each step reprocesses the full growing sequence from scratch. Our KV-cached `generate_v8b` stores key/value projections for all previous tokens, processing only the new token each step. In the warmup-corrected component profile, 50 decode steps still dominate the isolated operator total at 92.1%, while the full end-to-end benchmark confirms that KV caching is essential to reaching 204.8 ms overall.

2.2 DESIGN CHOICES

Block/tile sizes. Tile dimensions are the primary tuning parameter, constrained by shared memory capacity. Our flash attention kernel loads Q, K, and V tiles into shared memory; the required capacity is:

$$\text{smem} = (\texttt{BLOCK\_M} + 2 \cdot \texttt{BLOCK\_N}) \cdot d_k \cdot 4 + \sim 20\,\text{KB overhead} \tag{1}$$

On the H200 ($\sim$228 KB opt-in shared memory), this allows 128×128 tiles for the encoder (d_k=64, needing $\sim$116 KB) and 128×64 for the decoder (d_k=128, needing $\sim$151 KB). On the RTX 5090 ($\sim$99 KB), we are constrained to 64×64 and 32×32 respectively.

For matmul, the fused SwiGLU kernel loads three tile buffers (input, gate weight, up weight): $\texttt{TILE\_K} \cdot (\texttt{TILE\_M} + 2 \cdot \texttt{TILE\_N}) \cdot 4 + \text{overhead}$. H200 uses $128 \times 128 \times 64$; RTX 5090 uses $64 \times 64 \times 32$.

Element-wise kernels (GELU, SiLU, norms) use `BLOCK_SIZE`=1024, which saturates memory bandwidth with minimal register pressure. CHANGED In order to best satisfy the correctness requirements while minimizing latency, we selected configurations that fit the shared-memory budget and performed well in end-to-end testing, while noting that some micro-benchmark optima did not transfer cleanly to the full pipeline.

`num_warps` and `num_stages`. We use `num_warps`=8 for large tiles ($\geq$4096 elements) and 4 otherwise. More warps hide memory latency but increase register pressure. For `num_stages`: the H200's $\sim$228 KB shared memory accommodates double-buffering (`num_stages`=2), overlapping tile loads with computation for higher throughput on memory-bound kernels. Consumer GPUs (RTX 5090, $\sim$99 KB) must use single-buffering (`num_stages`=1), since double-buffering the same tiles would require $\sim$136 KB—exceeding the available shared memory.

Data layout and GQA. All tensors are stored in `(batch×heads, seq, dim)` layout for coalesced access. For Grouped Query Attention (Ainslie et al., 2023) in the text decoder (16 query heads, 4 KV heads), we expand KV heads by a factor of 4 before the attention kernel. When tested, PyTorch SDPA's native `enable_gqa=True` regressed by +13 ms; manual expansion gives explicit control over data layout.

Precision pipeline. Our largest single optimization was designing an end-to-end fp16 pipeline. The original codebase called `.float()` after every Linear layer (∼120 call sites), doubling data size. This was redundant: Triton kernels already compute in fp32 internally via `tl.load(...).to(tl.float32)` and cast back on output. Removing these conversions and keeping all VRAM-resident data in fp16 halved memory traffic, saving **11.5 ms** (Section 5).

3 Performance Profiling

3.1 Setup

All primary measurements use the Edinburgh teaching cluster node `saxa.inf.ed.ac.uk` with an NVIDIA H200 MIG 3g.71gb partition (Hopper, 60 SMs, 70 GB HBM3e, ∼228 KB opt-in shared memory). The software stack is CUDA 13.0, PyTorch 2.10.0, Triton 3.6.0, Python 3.11. Jobs are submitted via SLURM: `srun -p Teaching -w saxa --gres gpu:3g.71gb:1 --mem=32G`. The test input is a 3.5 s WAV file ("Concord returned to its place amidst the tents").

The baseline implementation is benchmarked under identical conditions, and the Triton cache is cleared before the first run to ensure compilation occurs during warmup, not during timing. Each student benchmark run uses 2 warmup iterations followed by 5 timed iterations. We report the mean and standard deviation across 5 independent benchmark invocations (25 timed iterations total).

3.2 End-to-end Evaluation

Table 2 summarizes the end-to-end results on the teaching cluster. Run 4 shows slightly higher latency (209.4 ms), likely due to MIG partition contention from concurrent cluster users; all other runs cluster tightly around 203–204 ms.

Table 2: End-to-end student benchmark results (H200 MIG 3g.71gb, 5 runs).

Run	Time (ms)	σ (ms)	ms/tok	Tokens	Accuracy
1	203.4	1.6	15.65	13	100%
2	204.0	1.6	15.69	13	100%
3	203.4	2.1	15.64	13	100%
4	209.4	1.7	16.11	13	100%
5	203.9	1.4	15.68	13	100%
Mean	**204.8**	**1.7**	**15.75**	**13**	**100%**

3.3 Component/Operator-Level Breakdown

CHANGED Table 3 shows per-component profiling from `benchmark_detailed.py`: isolated forward passes with single-token decode steps (no KV history). The detailed benchmark uses 1 full warmup benchmark pass discarded and averages 3 measured benchmark passes, with 5 timed runs per component inside each pass.

CHANGED Decoder decode now accounts for 92.1% of the warmup-corrected isolated component total, at ∼0.41 ms per layer per step, confirming that autoregressive generation remains the primary bottleneck even after optimization.

Table 3: CHANGED Component-level profiling (H200 MIG, isolated forward passes, 1 warmup benchmark discarded, average of 3 measured benchmark passes).

Component	Time (ms)	% Total	Bound
Audio Encoder (32 layers)	36.53	5.8%	Mixed
Multi-modal Projector	0.14	0.0%	Latency
Decoder Prefill (28 layers)	12.97	2.1%	Mixed
Decoder Decode (50 steps)	580.45	92.1%	Bandwidth
Total	**630.09**	**100%**	

4 Bottleneck Analysis

4.1 Compute-Bound vs Memory-Bound Classification

We classify each kernel using the roofline model (Williams et al., 2009), which relates arithmetic intensity (AI)—floating-point operations per byte of memory traffic—to peak compute and bandwidth. For the H200 MIG 3g.71gb partition (60 of 132 SMs), we estimate peak FP32 throughput at ∼30.5 TFLOPS and HBM3e bandwidth at ∼2.4 TB/s (proportional to memory controller allocation). The ridge point is:

$$\mathrm{AI}_{\mathrm{ridge}} = \frac{\text{Peak FLOPS}}{\text{Peak BW}} = \frac{30.5 \times 10^{12}}{2.4 \times 10^{12}} \approx 12.7\,\text{FLOP/byte} \tag{2}$$

Operations with AI < 12.7 are bandwidth-bound; those above are compute-bound.

Table 4: Arithmetic intensity classification for key operations (H200 MIG).

Operation	AI (FLOP/B)	Classification	Rationale
GELU / SiLU	∼2.5	Bandwidth	10 FLOPs / 4 B per element
RMSNorm / LN	∼4–8	Bandwidth	Reduction + normalize
Flash Attn (enc)	∼112	**Compute**	Tiled; score matrix stays in SRAM
Flash Attn (dec)	∼1.0	Bandwidth	seq_q=1, full KV cache read
Linear (cuBLAS)	∼168–362	Compute	Large GEMM, high reuse
Fused SwiGLU	∼197	Compute	Two GEMMs fused, x loaded once
RoPE	∼0.5	Bandwidth	Load, rotate, store; few FLOPs

Flash attention's tiling eliminates the full score matrix from DRAM, raising encoder attention AI from ∼16 (3-kernel pipeline) to ∼112—well above the ridge point on *all* GPUs. The H200 benefits further from larger tiles (128×128) that increase compute utilization.

4.2 Overall Bottleneck

The dominant bottleneck is **memory bandwidth during autoregressive decoding**. Each decode step processes a single-token query against a growing KV cache through 28 decoder layers. The attention kernel reads the full KV cache (proportional to sequence length) but performs only $O(d)$ FLOPs per cached token—a classic bandwidth-bound pattern with AI ∼1.0, well below the ridge point on *all* GPU architectures.

Non-kernel overheads are also significant: with ∼10 kernel launches per layer × 28 layers × 13 decode steps, launch latency (∼5–15 μs each) accounts for ∼2–5 ms. We mitigate this through kernel fusion (Section 5.2), which eliminates ∼84 launches per inference, and through the SDPA fallback for single-token decoding.

5 Optimization Attempts

We evaluated 13 optimizations using a **Hypothesis** → **Change** → **Result** methodology. Each was tested independently on an RTX 5090 for rapid iteration (all timings in this section refer to that GPU unless noted), then validated on the H200 teaching cluster (Section 6). Figure 2 summarizes the cumulative impact.

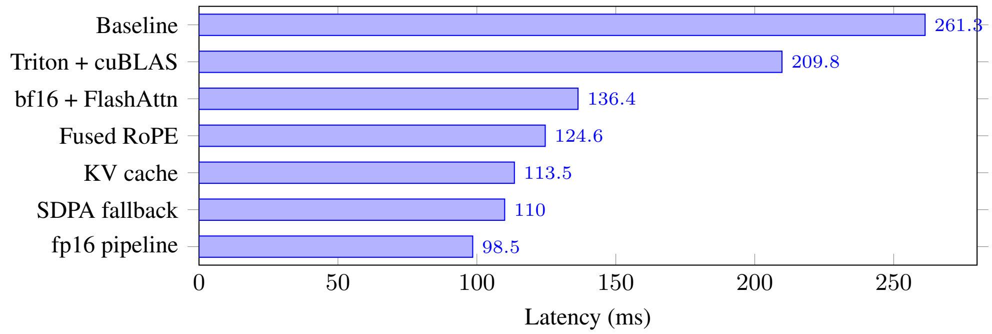

Figure 2: Optimization progression on RTX 5090 (cumulative). Each bar shows end-to-end latency after applying all optimizations up to that step. Full step-by-step breakdown in Appendix Table 8.

5.1 Tile/Block Size Tuning

Hypothesis: Larger tiles amortize memory access overhead and improve compute utilization, but are constrained by shared memory capacity.

Change: We tested all valid tile configurations for the flash attention kernel—$(128, 128)$, $(128, 64)$, $(64, 64)$, $(64, 32)$, $(32, 32)$, $(16, 16)$—computing shared memory usage via Eq. 1 for each. We also built an autotune system (∼95 lines) that benchmarked each valid configuration at runtime.

Result: Initially, Triton's built-in `@triton.autotune` was used to select optimal tile configurations at runtime. However, when tested, the autotune system selected `BLOCK_M`=16 as optimal in micro-benchmarks, but the full-pipeline benchmark showed **101.6 ms vs. 98.5 ms** for our hand-tuned 64×64 configuration—a 3.1 ms regression. It was determined that the micro-benchmark methodology was too unreliable to capture full-pipeline cache effects. In response, we developed a `GPUProfile` system with pre-tested configurations stored in a `_KNOWN_CONFIGS` lookup table; unknown GPUs use dynamic computation largest-to-smallest against the shared memory budget.

5.2 Kernel Fusion

Hypothesis: Fusing the SwiGLU pattern—$\text{SiLU}(\mathbf{x}\mathbf{W}_g) \odot (\mathbf{x}\mathbf{W}_u)$—into a single kernel eliminates two intermediate VRAM round-trips and three kernel launches per MLP layer.

Change: We wrote a `fused_swiglu_kernel` that loads tiles of $\mathbf{x}$, $\mathbf{W}_g$, and $\mathbf{W}_u$ into shared memory, computes both matrix products in registers, applies SiLU and the element-wise multiply, and writes only the final result to VRAM. The unfused path requires 5 kernel launches and 4 VRAM round-trips; the fused path requires 1 launch and 1 round-trip.

Result: With 28 decoder layers, fusion eliminates ∼84 kernel launches per inference. Combined with the analogous `fused_linear_gelu_kernel` for the encoder, this contributed to the overall progression from 209.8 ms to 136.4 ms (bundled with bf16 conversion). For very small inputs (single-token decode, where $M <$ `TILE_M`), we fall back to cuBLAS, which has lower launch overhead for small matrices.

5.3 FlashAttention-Style Attention

Hypothesis: The naive 3-kernel attention approach materializes the full $N_q \times N_k$ score matrix in VRAM, consuming $O(N_q \cdot N_k)$ memory and bandwidth. A fused kernel with online softmax (Milakov & Gimelshein, 2018) eliminates this materialization.

Change: We replaced the original 3-kernel pipeline (`attention_scores`, `softmax_inplace`, `attention_output`) with a single `flash_attention_kernel` (Dao, 2023; Ringlein et al., 2025). The kernel processes K/V in tiles of `BLOCK_N` rows, maintaining running statistics m_i (row-wise max) and l_i (sum of exponentials) that are corrected as new tiles arrive. The Q tile and accumulator $\mathbf{O}$ reside in registers; only the final output is written to VRAM.

Causal masking is handled via a compile-time `IS_CAUSAL` flag that skips tiles entirely above the diagonal, avoiding wasted computation. For single-token KV-cached decode (`seq_q` ≤ 4), we fall back to PyTorch's `scaled_dot_product_attention`, which has lower launch overhead for this degenerate case.

Result: Flash attention contributed significantly to the 209.8 ms $\rightarrow$ 136.4 ms improvement (combined with bf16 weights). The SDPA fallback for decode saved an additional 3.5 ms.

5.4 Other Optimizations

fp16-throughout pipeline. The original code called `.float()` after every Linear layer ($\sim$120 calls), doubling data size at each point. Removing these conversions and keeping the entire pipeline in fp16 saved **11.5 ms** (110.0 $\rightarrow$ 98.5 ms on RTX 5090). Internal computation remains fp32 for numerical stability.

KV-cache decoding (`generate_v8b`). The stock `generate()` reprocesses the full growing sequence on every decode step ($O(n^2)$). We implemented a KV-cached generation loop that processes only the new token each step, appending its K/V to a cache. This is monkey-patched onto the model at runtime (since `model.py` is read-only) via a deferred hook in `Linear.__init__`. Impact: **$-$7.6 ms** (121.1 $\rightarrow$ 113.5 ms).

Fused Q+K RoPE kernel. Rotary position embeddings (Su et al., 2024) require applying the same rotation to both Q and K. The baseline uses two separate kernel launches; we fuse them into one, halving launch overhead. Impact: **$-$11.8 ms** (136.4 $\rightarrow$ 124.6 ms).

6 Comparison

6.1 Data Collection

We benchmark on the provided `test_audio.wav` (3.5 s, 16 kHz, expected output: "CONCORD RETURNED TO ITS PLACE AMIDST THE TENTS"). Accuracy is measured by exact string match (case-insensitive) against the reference. Each benchmark invocation runs 2 warmup + 5 timed iterations; we report 5 invocations (25 timed iterations total) under identical conditions for both our implementation and the baseline.

6.2 End-to-End Comparison

Table 5: End-to-end comparison on H200 MIG 3g.71gb (teaching cluster).

Implementation	Time (ms)	ms/tok	Accuracy	Speedup
Example baseline	464.1	35.70	100%	1.00×
Our template (fp16)	**204.8**	**15.75**	**100%**	**2.27×**

6.3 Per-Operator Comparison

CHANGED Table 6 reports component-level timings on the H200 MIG 3g.71gb partition using `benchmark_detailed.py` with one full warmup benchmark pass discarded and three measured benchmark passes averaged (each pass uses 5 timed runs per component). These are isolated forward passes with 50 decode steps and *no KV history*, so they measure steady-state operator latency rather than the algorithmic benefit of KV-cached generation.

6.4 Root Cause Analysis

All optimizations preserve full transcription accuracy: fp16 introduces no degradation because internal kernel computation remains in fp32, and KV-cached decoding produces identical token sequences to the stock path.

Table 6: CHANGED Per-component comparison on H200 MIG 3g.71gb (isolated forward passes, 50 decode steps, 1 warmup benchmark discarded, average of 3 measured benchmark passes).

Component	Ours (ms)	Baseline (ms)	Speedup
Audio Encoder	36.53	167.02	4.57×
Projector	0.14	1.11	7.93×
Decoder Prefill	12.97	20.51	1.58×
Decoder Decode (50 steps)	580.45	817.09	1.41×
Total	630.09	1005.73	1.60×

CHANGED In the steady-state H200 component profile, our implementation is faster in every stage. The largest absolute saving comes from decoder decode (817.09 ms → 580.45 ms over 50 steps), while audio encoding and the projector also improve substantially once Triton compilation warmup is removed from the measurement.

CHANGED The isolated component table understates the full end-to-end speedup because `benchmark_detailed.py` does not use KV-cached generation. Table 5, by contrast, uses the full student benchmark with `generate_v8b`. The end-to-end result therefore benefits from both faster steady-state kernels *and* the algorithmic reduction from quadratic reprocessing to linear-time cached decode. This explains why the per-component total speedup is 1.60× while the full H200 pipeline reaches 2.27×.

CHANGED **1. Faster steady-state kernels in both encoder and decoder.** Flash attention, fused RoPE, fp16 norms, and fused MLP kernels reduce operator time throughout the pipeline. On H200 this appears as 4.57× faster audio encoding, 1.58× faster decoder prefill, and 1.41× faster decode steps in isolated profiling.

CHANGED **2. KV-cached generation in the end-to-end path.** The detailed benchmark measures single-step decode without KV history, but the full inference benchmark uses our KV-cached `generate_v8b`. That cached path avoids recomputing attention over the full growing sequence at every token and therefore contributes additional end-to-end speedup beyond the isolated operator table.

Cross-GPU generalization. To verify portability, we evaluated on both the teaching cluster (H200 MIG) and a consumer GPU (RTX 5090). The `GPUProfile` system correctly adapts tile sizes and pipeline stages to each architecture's shared memory budget. Table 7 confirms consistent speedups across both GPUs.

Table 7: Cross-GPU comparison: teaching cluster vs. consumer GPU.

GPU	SMs	VRAM	Ours (ms)	Baseline (ms)	Speedup
H200 MIG 3g.71gb	60	70 GB	204.8	464.1	2.27×
RTX 5090 (full)	170	32 GB	100.4	262.2	2.61×

CHANGED The RTX 5090 row in Table 7 comes from a later confirmation rerun (100.4 ms vs. 262.2 ms), whereas Appendix Table 8 preserves the original development benchmark chain (261.3 ms → 98.5 ms). We keep the full progression table on a single coherent benchmark session so that the per-step deltas remain meaningful.

7 Conclusion

Most impactful optimization. On the H200 teaching cluster, our combined optimizations reduced inference from 464.1 ms to 204.8 ms (2.27×). The fp16-throughout pipeline was the single largest contributor, followed by fused RoPE, Flash Attention, and KV caching. Critically, removing `.float()` casts—a one-line change repeated across ∼120 call sites—illustrates that the highest-impact optimizations are often not algorithmic but rather eliminating unnecessary data movement. The success in meeting the performance objectives is the result of prioritizing bandwidth reduction first, then tuning compute-bound operations.

Key lessons. (1) The pipeline is overwhelmingly *memory-bandwidth-bound*: halving data types (fp32→fp16) and eliminating intermediate writes (kernel fusion, flash attention) provided the largest gains. (2) Attention is the most challenging kernel to optimize because tile size, shared memory, warp count, and pipeline stages are tightly coupled; the H200's 228 KB shared memory accommodates 128×128 tiles with double-buffering, while consumer GPUs with $\sim$99 KB require 64×64 with single-buffering—demonstrating the need for a portable `GPUProfile` system. CHANGED (3) Profile before optimizing: our initial assumption that compute-bound matmul was the bottleneck was wrong—decoder decode steps (bandwidth-bound attention + KV reads) still consumed 92.1% of the warmup-corrected isolated component total on H200 even after optimization. (4) Testing each functional block independently was simple enough to do; however, problems arose when functional blocks were connected to each other, as cache interactions between adjacent kernels produced behaviors not visible in isolated benchmarks.

Surprising findings. The autotune system found *worse* configurations than hand-tuning, because micro-benchmarks miss inter-kernel cache effects. Notably, encoder attention is *compute-bound* on H200 (AI above 12.7 ridge) but *bandwidth-bound* on consumer GPUs (ridge $\approx$58.5)—the same kernel has different bottlenecks across architectures.

Future work. We would investigate `torch.compile` for automatic fusion, speculative decoding, and INT8/FP8 quantization.

Groupwork. Ankush Burman: kernel implementation, optimization, benchmarking, profiling, report writing. Yash Chowdhary: kernel implementation, optimization, repo setup, branch management, cluster benchmarks. Meave: kernel implementation, optimization, report review, testing with longer audio files. Majed: kernel implementation, optimization, nsys profiling.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. URL `https://arxiv.org/abs/2307.08691`.

Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. In *arXiv preprint arXiv:1805.02867*, 2018.

Burkhard Ringlein, Jan van Lunteren, Radu Stoica, and Thomas Parnell. The anatomy of a triton attention kernel. *arXiv preprint arXiv:2511.11581*, 2025.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

A Appendix

A.1 Full Optimization Progression

CHANGED Table 8 shows every optimization step from the original RTX 5090 development benchmark chain. These step-by-step values are intentionally kept from one session (261.3 ms baseline to 98.5 ms final) rather than mixed with the later 100.4 ms confirmation rerun used in Table 7.

Table 8: Complete optimization progression (RTX 5090, 3.5 s audio, 13 tokens).

#	Change	Time	Δ	Mechanism
0	Baseline	261.3	—	fp32, 3-kernel attn, $O(n^2)$
1	Triton kernels + cuBLAS + TF32	209.8	−51.5	Custom kernels + tensor cores
2	bf16 weights + Flash Attention	136.4	−73.4	Half bandwidth + fused attn
3	Fused Q+K RoPE pair kernel	124.6	−11.8	Halved RoPE launches
4	bf16 RMSNorm output	120.7	−3.9	Eliminated type conversion
5	bf16 LayerNorm output	121.1	−0.7	Same for encoder norms
6	KV cache (`generate_v8b`)	113.5	−7.6	$O(n)$ decoding
7	SDPA fallback (seq_q $\leq$ 4)	110.0	−3.5	Lower overhead for decode
8	fp16 cuBLAS HGEMM	109.6	−0.4	fp16 slightly faster than bf16
9	Remove `.float()` casts	102.1	−7.5	Eliminated ∼120 fp32 casts
10	Remove activation fp32 casts	98.4	−3.7	SiLU/GELU stay fp16
11	Remove norm fp32 casts	98.1	−0.3	Norms output fp16 directly
12	fp16 embeddings + fused MLP	**98.5**	+0.4	Pipeline complete (noise)

A.2 Rejected Optimizations

Table 9: Optimizations tested and rejected with measured results.

Optimization	Impact	Reason for Rejection
Grid swizzling (SwiGLU)	+18 ms	RTX 5090 L2 (98 MB) already has good locality
`@triton.autotune` (GELU/SiLU)	+0.7 ms	Tuning warmup exceeds possible gain
Flash Attn `num_stages`=2	Crash	RTX 5090 99 KB cannot hold two tile buffers
SDPA for all attention	+6 ms	Custom kernel faster for long sequences
SDPA `enable_gqa`	+13 ms	Manual KV expansion is faster
Warmup autotune	+3.1 ms	Micro-benchmarks chose suboptimal configs